Telas finais apresentadas do
Simulador de SO Multitarefa

João Vitor Bernardi
RA: 2699559

Lais Sayoko Toda Duarte
RA:2699567

gitHub para repositório: https://github.com/joaobernardi2006/Simulador-de-Sistema-Operacional-Multitarefas.git

Após a finalização da implementação de todas as funções e requisitos solicitados para o Simulador de um Sistema Operacional Multitarefas, temos diversos elementos visuais, que são apresentados tanto no terminal durante a execução da simulação, quanto ao fim da mesma, no arquivo de tipo ***.svg.***

1. Telas apresentadas no Terminal

1.1. Tela de definição do arquivo par a simulação

Valores padrões

```
Valores padrao do sistema:
  Algoritmo : SRTF
  Quantum   : 2 tick(s)
  CPUs      : 5
(Esses valores sao sobrescritos pelo arquivo de configuracao)

Digite o nome do arquivo de configuracao:
```

Nessa primeira tela que aparece logo após inicializar a simulação, é solicitado com que o usuário escolha o arquivo ***.txt*** do qual será utilizado para a ocorrência da simulação e consequentemente da formação do diagrama de Gantt. O arquivo necessariamente precisa estar no mesmo diretório que os arquivos de execução do simulador e seu formato deve seguir o padrão já explicitado no documento de requisitos:

algoritmo_escalonamento;quantum;qtde_cpus
id;cor;ingresso;duracao;prioridade;lista_eventos

Assim, temos neste arquivo o algoritmo de escalonamento que será utilizado pelo simulador (SRTF OU PRIOp), o quantum para as tarefas, a quantidade de CPUs que a simulação deve trabalhar, o id da tarefa, bem como sua cor, duração, prioridade de execução (essencial para o PRIOp) e a lista de eventos, respectivamente.
Temos também no terminal os valores padrões que já foram definidos inicialmente pelo próprio.

Esses elementos padrões definidos pelo sistema derivam das funções ***"inicializaConfigSistema()",*** que define-os. Agora, a leitura do arquivo é feita pela função "l***erConfiguracao()***"

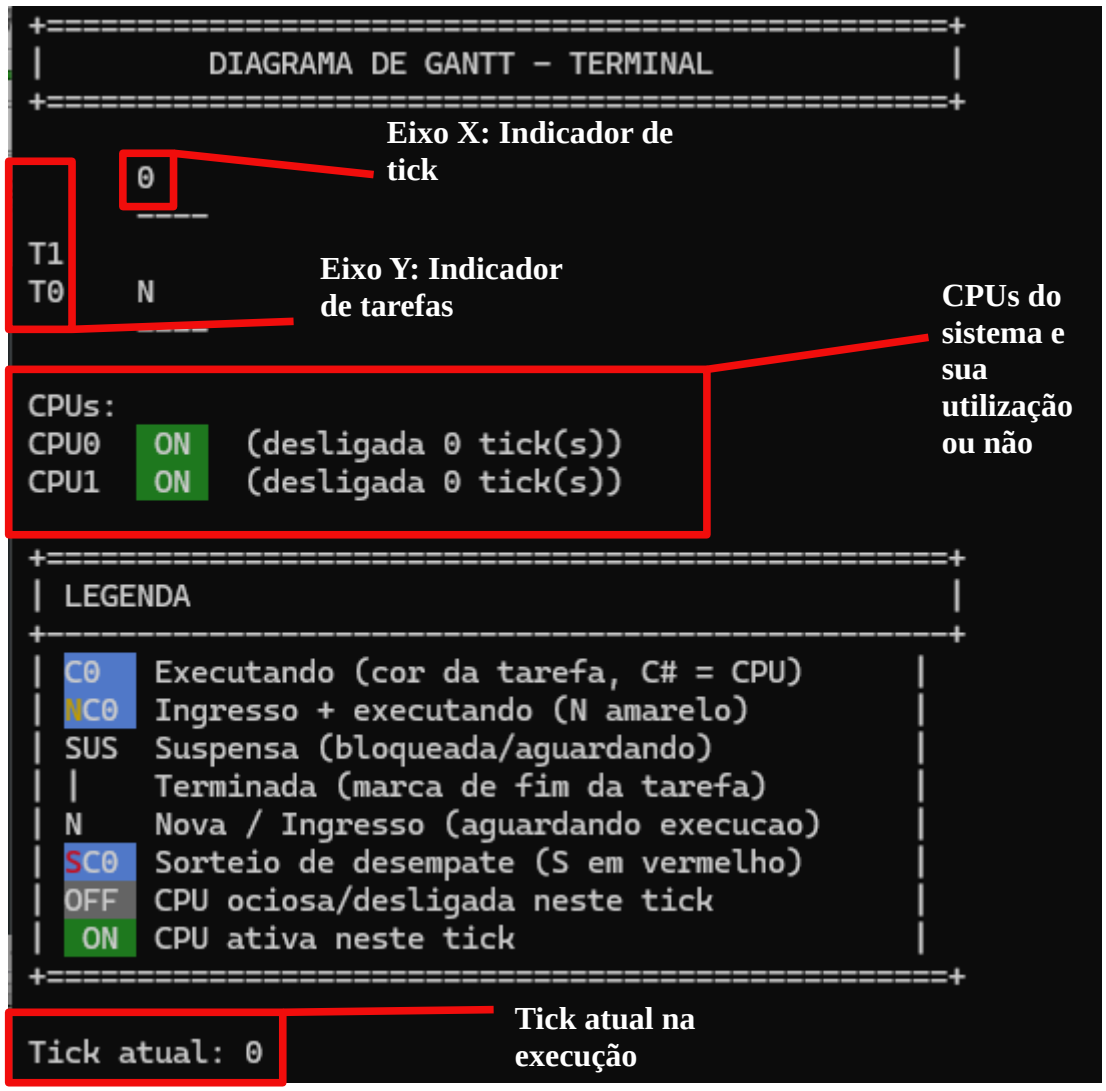
1.2. Tela de escolha do modo de execução

```
Valores padrao do sistema:
  Algoritmo : SRTF
  Quantum   : 2 tick(s)
  CPUs      : 5
(Esses valores sao sobrescritos pelo arquivo de configuracao)

Digite o nome do arquivo de configuracao: teste.txt
Como deseja realizar a simulacao? Passo-a-passo(a) ou completa(b)?
```

Após essa escolha do arquivo que define as infomações do simulador, temos agora uma mensagem que busca identificar qual é o modo de execução a ser seguido, podendo ser passo a passo (semelhantemente a um debugger) e a execução completa (A simulação é realizada até o final, sem interrupções). Caso o usuário digite a letra “A”, o simulador entende que o modo de execução selecionado é o passo a passo, e caso o usuário digite “B”, temos a escolha da execução completa.
Independentemente do usuário digitar letra minúscula ou maiúscula, é realizado o tratamento para garantir com que seja algo adequado. Se for digitado um caractere aleatório, é exibido um erro.
Essa escolha para saber qual deve ser o modo de execução é feita diretamente da função ***“main()”,*** onde caso seja optado por seguir a opção Passo-a-Passo, é feito uma chamada para a função “***execucaoPassoAPasso()***”, caso seja escolhido a opção de execução completa, a função chamada é ***“execucaoCompleta()”***

```
Opcao invalida!
Process returned 1 (0x1)   execution time : 5.538 s
Press any key to continue.
```

1.3. Execução Passo a Passo



Ao optar pela execução passo a passo (a), o usuário depara-se com esta tela, onde temos a estrutura base do diagrama de Gantt. Ao topo, temos o título do diagrama e logo abaixo, no eixo Y, temos a indicação das tarefas crescentemente ordenadas de baixo para cima. O eixo X indica a evolução dos ticks. Abaixo do diagrama, temos um indicador para quando um determinado CPUx está ligado (ON) ou não (OFF) e ao seu lado, temos um contador que indica a quantidade de ticks com que este contador permaneceu desligado. Por fim, temos uma legenda que indica o que cada símbolo que é apresentado no diagrama representa e o tick atual no qual o simulador se encontra.

O diagrama gerado no terminal é feito por meio de ***"imprimeGanttTerminal()",*** utilizando majoritariamente a variável ***DiagramaGantt** diagrama,*** matriz que armazena o estado da tarefa, a CPU usada e se foi sorteado ou não.
Por sua vez, os estados das tarefas são definidos por meio de ***#define,*** e cada estado particular das tarefas é armazenado por uma variável ***int estado,*** presente na struct ***TCB.*** Nessa mesma struct, temos armazenado: ID, estado, duração, prioridade, tempo restante, tempo de espera, cor, quantum, eventos.
Ademais, as informações referentes as CPUs em uso e seus respectivos tempos de ociosidade são mantidas na struct ***CPU.*** O contado de ociosidade da CPU é uma variável: ***CPUs[i].tempo_desligado++;***
Por fim, o tick atual é derivado de uma variável de nome ***int tick,*** sendo também armazenado em ***EstadoSistema.*** A sua exibição é feita pelo comando ***printf("Tick atual: %d\n\n", atual_Estado->tick);***

Trecho:

```
A - Avancar
R - Retroceder
M - Modificar tarefa
S - Sair

>
```

Partindo propriamente para as opções das quais o usuário pode escolher para executar na simulação, temos:

1.3.1 Avançar(A):

Caso o usuário escolha avançar, o simulador executará um tick, indo diretamente para o seguinte. Isso ocorre através de uma chamada de função dentro da função responsável pela execução passo a passo.

```
+==================================================+
|          DIAGRAMA DE GANTT - TERMINAL            |
+==================================================+

      0   1
      --------
T1
T0    NC0
      --------

CPUs:
CPU0   ON  ON   (desligada 0 tick(s))
CPU1  OFF  ON   (desligada 1 tick(s))

+==================================================+
| LEGENDA                                          |
+--------------------------------------------------+
| C0   Executando (cor da tarefa, C# = CPU)        |
| NC0  Ingresso + executando (N amarelo)           |
| SUS  Suspensa (bloqueada/aguardando)             |
| |    Terminada (marca de fim da tarefa)          |
| N    Nova / Ingresso (aguardando execucao)       |
| SC0  Sorteio de desempate (S em vermelho)        |
| OFF  CPU ociosa/desligada neste tick             |
|  ON  CPU ativa neste tick                        |
+==================================================+

Tick atual: 1
```

Tarefa em execução

Legenda do diagrama

Temos então, o avanço para o tick seguinte da simulação, onde algumas situações referentes as tarefas foram atualizadas, por exemplo, a tarefa T0 entrou na CPU 0 e está sendo executada.
Essa função de avançar é responsável por chamar a função ***"executaTick()"*** que, como o nome diz, realiza a execução de um tick. Nessa mesma função, é chamada "***simular_tick()***", responsável por modificar as tarefas e as CPUs, alterar estados e atualizar o diagrama de Gantt

1.3.2. Retroceder(R):
Nessa opção, o simulador retorna para o tick imediatamente anterior, restaurando o estado que as tarefas e as CPUs se encontravam, incluindo o tempo restante para o término da tarefa, sua duração e sua representação gráfica no diagrama de Gantt.
Como o tick, o retroceder deriva da struct ***EstadoSistema***, que semelhantemente a uma lista, armazena "snapshots" do sistema, onde ponteiros são responsáveis por permitir o retrocesso (ponteiro para o snapshot anterior) ou o avanço (ponteiro para o snapshot seguinte). Isso é bem perceptivel no comando: ***atual_Estado = atual_Estado->anterior;***

1.3.3 Modificar (M):
Agora, caso o usuário digite "M", a opção escolhida pelo mesmo permite com que ele realize a mudança de estado das tarefas no tick atual. Para isso, ele é novamente direcionado para outra tela.
Ainda na função "***execucaoPassoAPasso***", a opção de modificar o estado utiliza uma variável que armazena o estado escolhido e então atualiza o estado da tarefa. Além disso, ao optar pela modificação do estado, o simulador também, se necessário conforme o estado modificado, remove da fila; remove da CPU, recoloca na fila, encerra tarefa e atualiza contexto.

```
Tick atual: 2

A - Avancar
R - Retroceder
M - Modificar tarefa
S - Sair

> m

Tarefas disponiveis:
  ID 0 | Estado atual: PRONTA | Restante: 4
  ID 1 | Estado atual: NOVA | Restante: 4

Digite o ID da tarefa a modificar:
```

Informações referentes as tarefas

Nessa tela, é apresentado as tarefas disponíveis na execução, bem como seus respectivos Ids, o estado no qual se encontram e o tempo restante de execução delas.
Para avançar o usuário escolhe o id da tarefa que deseja modificar

```
Digite o ID da tarefa a modificar: 0

Escolha o novo estado para a Tarefa 0:
  1 - NOVA
  2 - PRONTA
  3 - EXECUTANDO
  4 - TERMINADA
  5 - SUSPENSA
Digite o numero do estado:
```

Assim, uma nova tela é apresentada para o usuário, agora apresentando as opções que se tem para modificar o estado da tarefa anteriormente selecionada. Assim, aquele que manipula a simulação precisa digitar o número referente a tarefa, como indicado no terminal. Estes números são os mesmos utilizados para definir internamente cada estado, portanto, caso haja a inserção de um número não presente nas opções, uma mensagem de erro é indicada e não é realizada nenhuma mudança

```
! Estado invalido. Nenhuma alteracao realizada.
```

Ademais, caso seja selecionado um número válido e consequentemente um estado novo, a função passo a passo altera o estado da tarefa e realiza o tratamento da mesma, realizando as alterações necessárias para com que a mudança seja verdadeiramente aplicada em toda a tarefa, influenciando todos os seus atributos e sua representação gráfica, e não somente mude o valor interno do estado dela.

```
+==================================================+
|          DIAGRAMA DE GANTT - TERMINAL            |
+==================================================+

      0   1   2   3
      ----------------
T1            N
T0    NC0 |
      ----------------

CPUs:
CPU0   ON  ON  ON  ON   (desligada 0 tick(s))
CPU1  OFF  ON  ON  ON   (desligada 1 tick(s))

+==================================================+
| LEGENDA                                          |
+--------------------------------------------------+
| C0   Executando (cor da tarefa, C# = CPU)        |
| NC0  Ingresso + executando (N amarelo)           |
| SUS  Suspensa (bloqueada/aguardando)             |
| |    Terminada (marca de fim da tarefa)          |
| N    Nova / Ingresso (aguardando execucao)       |
| SC0  Sorteio de desempate (S em vermelho)        |
| OFF  CPU ociosa/desligada neste tick             |
|  ON  CPU ativa neste tick                        |
+==================================================+

Tick atual: 1
```

Indicação que a tarefa foi finalizada

Neste exemplo, modifiquei o estado da tarefa T0 para terminada, então a tarefa é devidamente encerrada

1.3.4. Sair(S):

```
Tick atual: 1

A - Avancar
R - Retroceder
M - Modificar tarefa
S - Sair

> S

Ociosidade por CPU:
  CPU 0: desligada por 0 tick(s)
  CPU 1: desligada por 1 tick(s)

Process returned 0 (0x0)   execution time : 1482.975 s
Press any key to continue.
```

Por fim, caso o usuário escolha “Sair”, a simulação é encerrada no tick atual e apresenta a ociosidade, ou seja, o tempo de desligamento total, que cada CPU teve durante a execução.

1.4. Execução Completa

CPU que está em uso pela tarefa

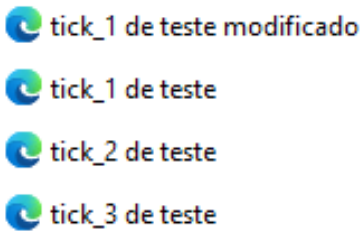
```
+==================================================+
|          DIAGRAMA DE GANTT - TERMINAL            |
+==================================================+

      0   1   2   3   4   5   6
      ---------------------------
T1            NC0 C0  C0  C0  |
T0    NC0 C0  C1  C1  C1  C1  |
      ---------------------------

CPUs:
CPU0   ON  ON  ON  ON  ON  ON  ON   (desligada 0 tick(s))
CPU1  OFF OFF  ON  ON  ON  ON  ON   (desligada 2 tick(s))

+==================================================+
| LEGENDA                                          |
+--------------------------------------------------+
| C0   Executando (cor da tarefa, C# = CPU)        |
| NC0  Ingresso + executando (N amarelo)           |
| SUS  Suspensa (bloqueada/aguardando)             |
| |    Terminada (marca de fim da tarefa)          |
| N    Nova / Ingresso (aguardando execucao)       |
| SC0  Sorteio de desempate (S em vermelho)        |
| OFF  CPU ociosa/desligada neste tick             |
|  ON  CPU ativa neste tick                        |
+==================================================+


Process returned 0 (0x0)   execution time : 7.447 s
Press any key to continue.
```

Por outro lado, caso o usuário escolha realizar a execução completa(b), a função que é chamada na main é a referente a este modo de execução. Será apresentado no terminal somente a versão final do diagrama, não permitindo modificações ou retrocessos. Também é apresentada todo o uso das CPUs e o tempo de ociosidade total ao seu lado

2. Diagrama .svg

Em ambos modos de execução, é criado um arquivo ***.svg*** no mesmo diretório que se encontra o simulador. Este arquivo apresenta o diagrama de Gantt da execução.

No caso da execução passo a passo, é gerado um diagrama para cada tick executado, seja um avanço ou um retrocesso, e as modificações realizadas são igualmente apresentadas, tendo em seu nome *"modificado"*



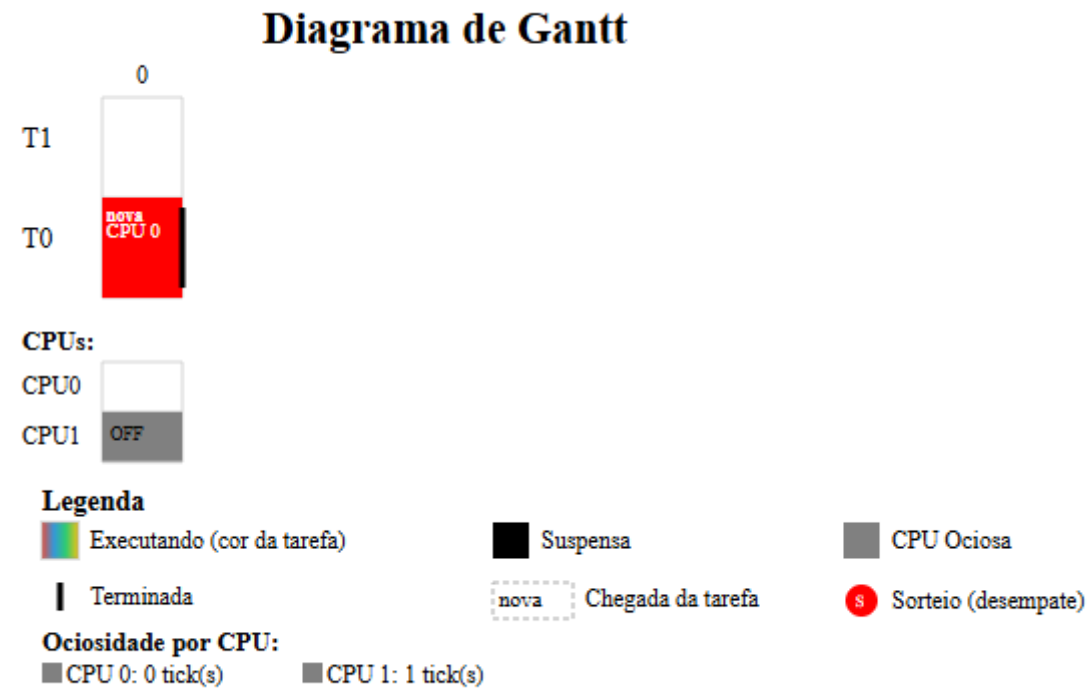
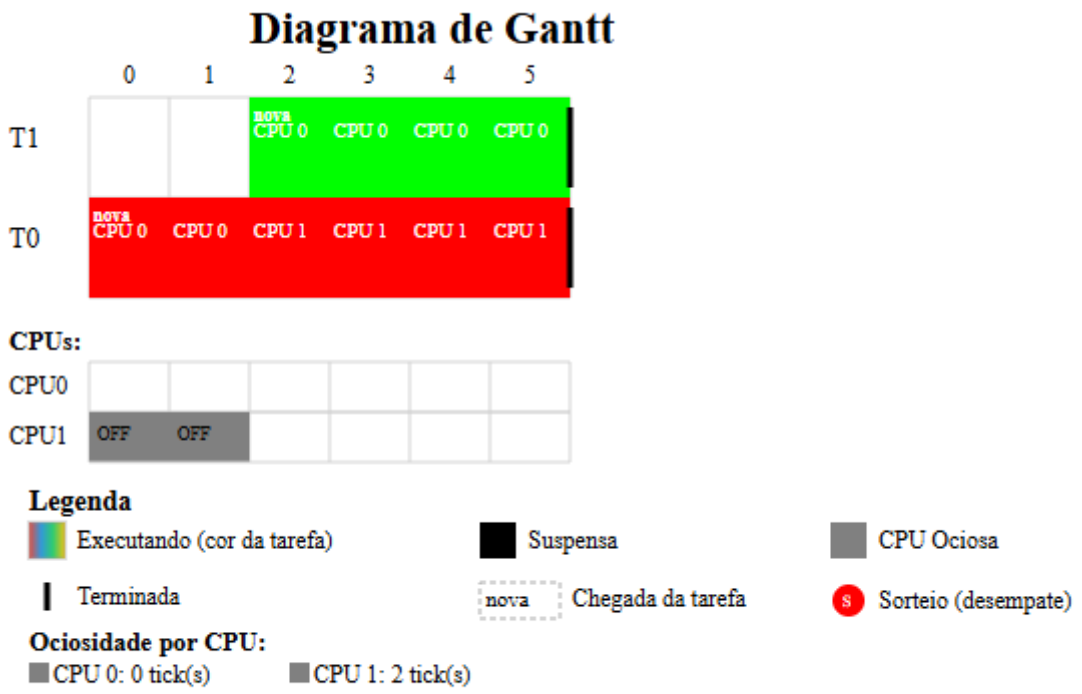


Diagrama de Gantt da modificação apresentada em 1.3.3

Na execução completa, é gerado apenas o diagrama de Gantt final, onde já foi realizado toda a simulação.



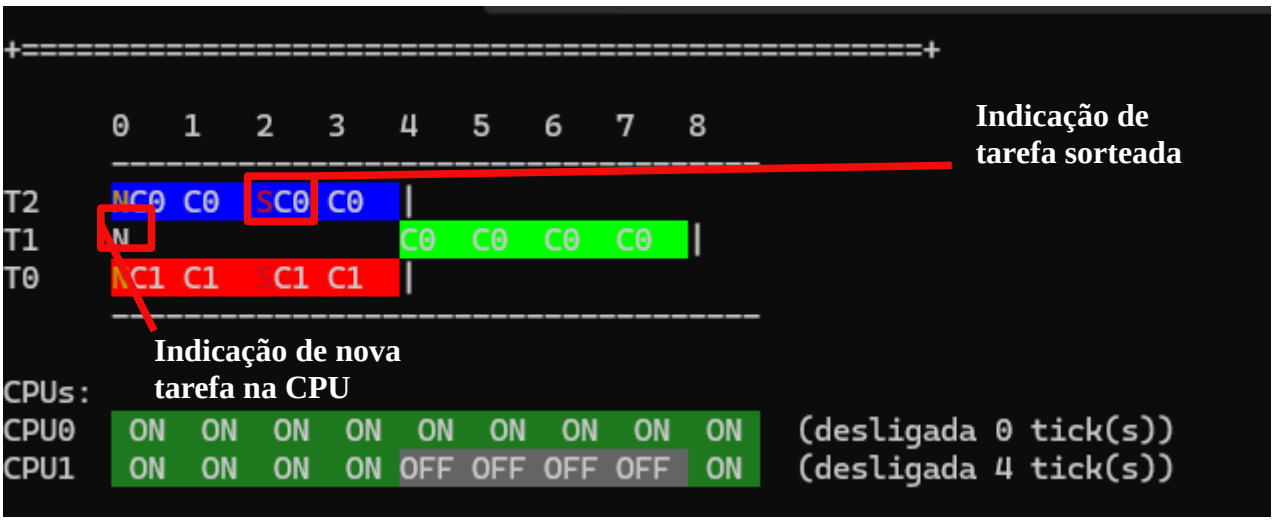
Nesse diagrama de Gantt, é apresentado cada CPU que está sendo utilizada por determinada tarefa em determinado tick, bem como uma legenda que indica cada um dos casos que a tarefa pode se encontrar.

Esses diagramas são gerados pela função ***"criaSVG()",*** e por padrão, o arquivo vem com o nome no formato ***config.nome_arquivo***

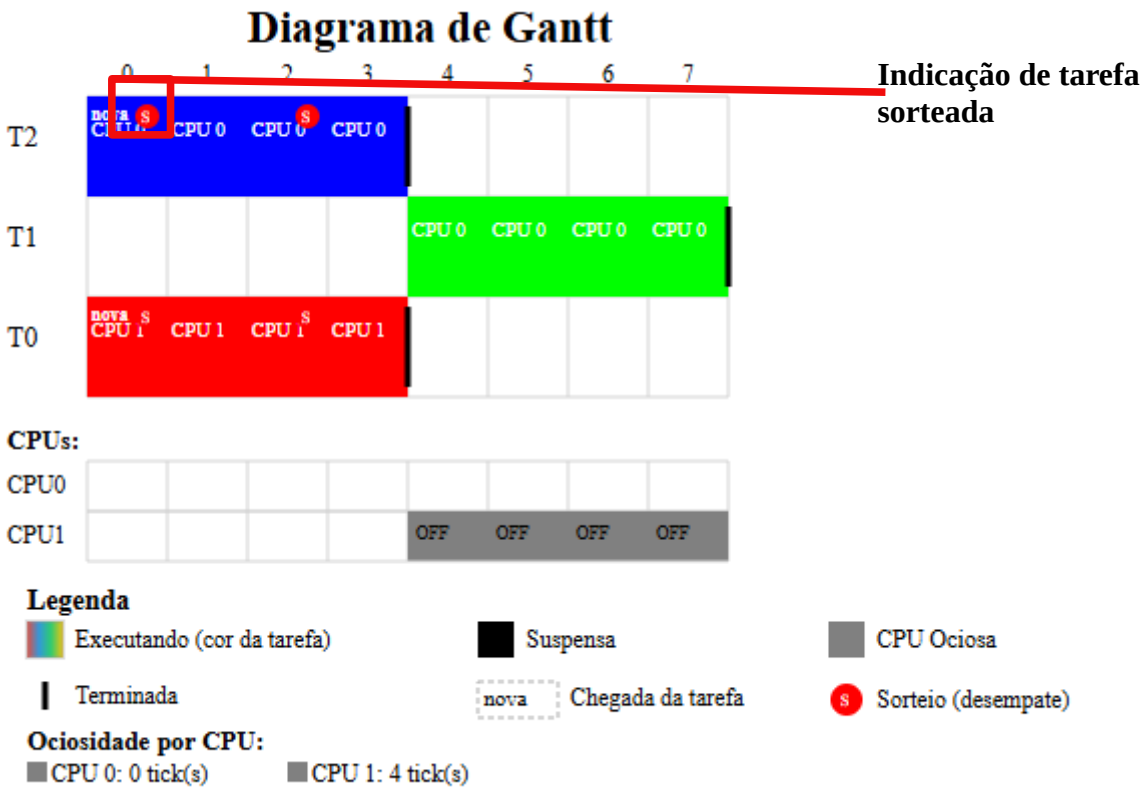
3. Casos especiais

3.1. Tarefa sorteada

Como definido em ambas opções de escalonamento, temos como último critério de desempate um sorteio para saber qual tarefa será a próxima a utilizar a CPU em questão. Dessa forma, tem-se um elemento visual para indicar que ela foi escolhida via sorteio, como indica as legendas do terminal e do diagrama de Gantt no formato ***.svg:***



No terminal, temos um indicador ***S*** ao lado da CPU que está em uso para representar que o critério de desempate utilizado para a escolha da tarefa a executar foi um sorteio.



No diagrama de Gantt gerado no formato ***.svg,*** o indicador da utilização do sorteio como critério de desempate é apresentado como um pequeno S dentro de um circulo vermelho na tarefa sorteada.

Essa sinalização de tarefa sorteada é feia utilizando o campo ***int sorteio*** da struct ***DiagramaGantt***

De maneira geral, os dados essenciais para o funcionamento da simulação são obtidos de "***ConfigSistema lerConfiguracao()",*** *responsável pela leitura dessas informações no arquivo* ***.txt*** disponibilizado.

4.Conclusão

Assim, temos que essa simulação é um claro exemplo de um programa complexo e com diversas interações que ocorrem entre funções. Essa comunicação é essencial para o funcionamento do simulador, bem como é necessário mantê-las bem definidas e claras para tornar a compreensão do código mais fácil e acessível a novos programadores e/ou indivíduos que estão querendo apenas utilizar o software. Para tal, também foi realizado diversos comentários por todo o código para torná-lo mais legível